

Analyses: Swabian Aurignacian

Description

This file enables analyses of feature values for sequences on Swabian Aurignacian objects (entropy rate, TTR, repetition rate, unigram entropy) as a function of meta information (object type, volume, cave site, etc.).

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"
## [1] "x86_64-pc-linux-gnu"
```

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`.

```
library(ggplot2)
library(ggrepel)
library(ggExtra)
library(gridExtra)
library(ggpubr)
library(ggribes)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v lubridate  1.9.3      v tibble    3.2.1
## v purrr      1.0.2      v tidyr     1.3.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::combine() masks gridExtra::combine()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(performance)
library(see)
library(GGally)
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg      ggplot2
```

```
library(MASS)
```

```
##  
## Attaching package: 'MASS'  
##  
## The following object is masked from 'package:dplyr':  
##  
##      select
```

```
library(forcats)
```

```
library(jtools) # for effect plots by predictor variable
```

Give the package versions.

```
##      jtools      MASS      GGally      see performance      lubridate  
## "2.2.2" "7.3-60" "2.2.1" "0.8.4" "0.12.1" "1.9.3"  
## forcats      stringr      dplyr      purrr      readr      tidyr  
## "1.0.0" "1.5.1" "1.1.4" "1.0.2" "2.1.5" "1.3.1"  
## tibble      tidyverse      ggridges      ggpubr      gridExtra      ggExtra  
## "3.2.1" "2.0.0" "0.5.6" "0.6.0" "2.3" "0.10.1"  
## ggrepel      ggplot2  
## "0.9.5" "3.5.1"
```

Load Data

Load feature estimations with meta data for the Aurignacian mobile objects.

```
sign.data <- read.csv("~/Github/PaleoSigns/results/features/signBase_features.csv")  
# select Aurignacian data  
sign.data <- sign.data[sign.data$archaeological_unit == "Aurignacien", ]
```

Simple Stats

```
# number of sequences/objects  
nrow(sign.data)
```

```
## [1] 211
```

```
# number of sign tokens  
sum(sign.data$num.tokens)
```

```
## [1] 3000
```

```
# cave sites  
table(sign.data$site_name)
```

```
##  
##      Bockstein-Törle      Geissenklösterle      Hohle Fels      Hohlenstein-Stadel  
##              1              28              33              2  
##      Vogelherd  
##              147
```

```
# material  
table(sign.data$material)
```

```
##
## animal tooth      antler      bone      fossil      ivory      sandstone
##           1           6          67          1         134          1
##      slate
##           1

# object_type
table(sign.data$object_type)

##
##      antler object      awl      bâton percé
##           3           4           3
##      figurine figurine anthropomorph      figurine zoomorph
##           6           6           21
##           flute      personal ornament      plaquette
##           8           16           1
##      possible figurine      possible flute      rod/bâton
##          20           15           15
##      spatula/lissoir      tube      tube/flute
##          10           1           20
##      undetermined
##          62
```

Preprocessing

Add columns for object volume and dates.

```
# volume in cm^3 (mm multiplications divided by 1000)
sign.data$volume <- (sign.data$length_mm*sign.data$width_mm*sign.data$depth)/1000
sign.data$date <- as.numeric(substr(sign.data$date_bp_max, 1, 5))
```

Collapse Factors

Collapse some factors of meta information which are uncommon and/or related. Note that using the factor levels as is will lead to problems with model fitting, since there are sometimes very few data points per level.

```
# material: collapse manually
sign.data$material <- fct_collapse(sign.data$material,
  ivory = "ivory",
  antler = "antler",
  bone = "bone",
  other_level = "other")

# object_type: collapse manually
sign.data$object_type <- fct_collapse(sign.data$object_type,
  figurine_anthropomorph = c("figurine anthropomorph",
    "plaquette"),
  # note: the "plaquette" is the so-called Adorant with an anthropomorph figure on it.
  figurine_zoomorph = c("figurine zoomorph"),
  tool = c("awl", "bâton percé",
    "rod/bâton", "spatula/lissoir"),
  tube_flute = c("flute", "tube", "tube/flute"),
```

```

personal_ornament = c("personal ornament"),
other_level = "other")

```

Exclude certain levels of columns to check the robustness of results.

```

#sign.data <- sign.data[sign.data$object_type != "other" &
#                        sign.data$material != "other" , ]
#sign.data <- sign.data[sign.data$coding_certainty != "Uncertain", ]
# sign.data <- sign.data[sign.data$num.tokens < 100, ]

# number of sequences/objects after preprocessing
nrow(sign.data)

```

```
## [1] 211
```

Simple Statistics Plots

```

# barplot for object counts per cave site
site.barplot <- ggplot(sign.data, aes(x = site_name, fill = site_name)) +
  geom_bar() +
  ylim(0, 200) +
  geom_text(aes(label = after_stat(count)), stat = "count", vjust = -2.5, hjust = 0.5) +
  geom_text(aes(label = paste(round(after_stat(prop)*100, 1), "%")),
            stat = "prop", vjust = -1, hjust = 0.5) +
  theme(legend.position = "none") +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
  xlab("Cave Site")
#coord_flip()
#site.barplot

# barplot for object counts per material
material.barplot <- ggplot(sign.data, aes(x = material, fill = material)) +
  geom_bar() +
  ylim(0, 200) +
  geom_text(aes(label = after_stat(count)), stat = "count", vjust = -2.5, hjust = 0.5) +
  geom_text(aes(label = paste(round(after_stat(prop)*100, 1), "%")),
            stat = "prop", vjust = -1, hjust = 0.5) +
  theme(legend.position = "none") +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
  xlab("Material")
#coord_flip()
#material.barplot

# barplot for object counts per object_type
type.barplot <- ggplot(sign.data, aes(x = object_type, fill = object_type)) +
  geom_bar() +
  ylim(0, 200) +
  geom_text(aes(label = after_stat(count)), stat = "count", vjust = -2.5, hjust = 0.5) +
  geom_text(aes(label = paste(round(after_stat(prop)*100, 1), "%")),
            stat = "prop", vjust = -1, hjust = 0.5) +
  theme(legend.position = "none") +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +

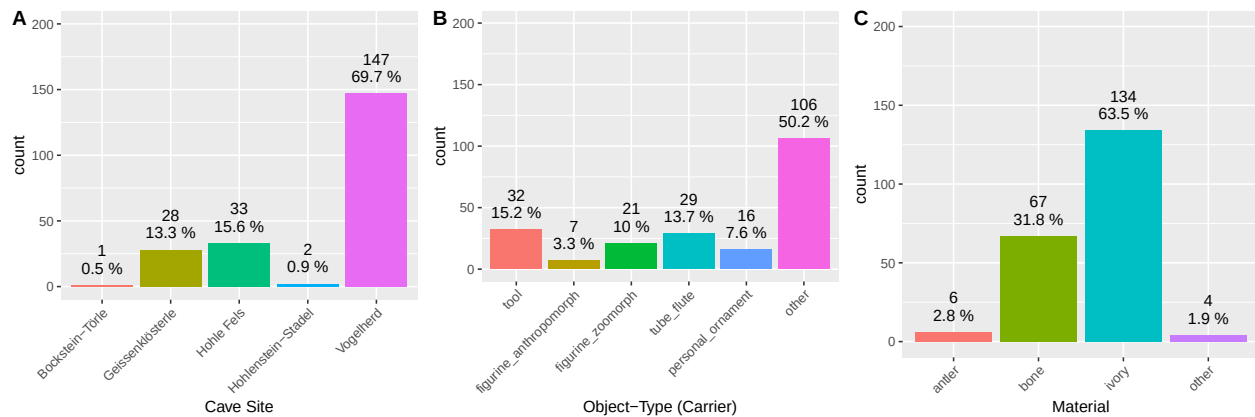
```

```
xlab("Object-Type (Carrier)")
#coord_flip()
#type.barplot
```

Figures Combined

Combine figures.

```
combined.barplot <- ggarrange(site.barplot, type.barplot, material.barplot,
                              ncol = 3, nrow = 1,
                              labels = "AUTO")
combined.barplot
```



Safe complete figure to file.

```
ggsave("~/Github/PaleoSigns/figures/barPlots_Aurignacian.pdf",
        combined.barplot, dpi = 300, width = 12, height = 4, device = cairo_pdf)
```

Sign Stats

Cave Sites

Get frequency counts for individual sign types per cave site.

```
# initialize dataframe to append results to
caveSigns.df <- data.frame(signs = character(0),
                           Freq = numeric(0),
                           site_name = character(0))

for (site_name in unique(sign.data$site_name)){
  # get all cleaned codings per cave
  codings <- sign.data[sign.data$site_name == site_name, ]$coding_clean
  # collapse to one sequence
  codings.collapsed <- paste(codings, collapse = "")
  # remove underscores
  codings.collapsed <- gsub("_", "", codings.collapsed)
  # split into individual signs
  signs <- unlist(strsplit(codings.collapsed, ""))
  # get data frame with frequency counts
```

```

signs.df <- as.data.frame(table(signs))
# order by frequency
signs.df <- signs.df[order(-signs.df$Freq), ]
# add column with site_name
signs.df$site_name <- rep(site_name, nrow(signs.df))
# append results to dataframe
caveSigns.df <- rbind(caveSigns.df, signs.df)
}

```

Barplot for frequency counts per sign type and cave site.

```

caveSigns.barplot <- ggplot(caveSigns.df, aes(x = reorder(signs, Freq),
                                                  y = Freq, fill = signs)) +

  #ylim(0, 700) +
  geom_text(aes(label = signs), hjust = -1) +
  geom_bar(stat = "identity") +
  facet_wrap(~site_name, ncol = 5) +
  theme(legend.position = "none") +
  coord_flip() +
  xlab("Sign Types") +
  ylab("Token Frequency")
#caveSigns.barplot

```

Object Type

Get frequency counts for individual sign types per object type.

```

# initialize dataframe to append results to
objectSigns.df <- data.frame(signs = character(0),
                              Freq = numeric(0),
                              object_type = character(0))

for (object_type in unique(sign.data$Object_type)){
  # get all cleaned codings per object type
  codings <- sign.data[sign.data$Object_type == object_type, ]$coding_clean
  # collapse to one sequence
  codings.collapsed <- paste(codings, collapse = "")
  # remove underscores
  codings.collapsed <- gsub("_", "", codings.collapsed)
  # split into individual signs
  signs <- unlist(strsplit(codings.collapsed, ""))
  # get data frame with frequency counts
  signs.df <- as.data.frame(table(signs))
  # order by frequency
  signs.df <- signs.df[order(-signs.df$Freq), ]
  # add column with site_name
  signs.df$object_type <- rep(object_type, nrow(signs.df))
  # append results to dataframe
  objectSigns.df <- rbind(objectSigns.df, signs.df)
}

```

Barplot for frequency counts per sign type and cave site.

```

objectSigns.barplot <- ggplot(objectSigns.df, aes(x = reorder(signs, Freq),
                                                    y = Freq, fill = signs)) +

  #ylim(0, 500) +
  geom_text(aes(label = signs), hjust = -1) +
  geom_bar(stat = "identity") +
  facet_wrap(~object_type, ncol = 6) +
  theme(legend.position = "none") +
  coord_flip() +
  xlab("Sign Types") +
  ylab("Token Frequency")
#objectSigns.barplot

```

Material

Get frequency counts for individual sign types per material.

```

# initialize dataframe to append results to
materialSigns.df <- data.frame(signs = character(0),
                                Freq = numeric(0),
                                material = character(0))

for (material in unique(sign.data$material)){
  # get all cleaned codings per cave
  codings <- sign.data[sign.data$material == material, ]$coding_clean
  # collapse to one sequence
  codings.collapsed <- paste(codings, collapse = "")
  # remove underscores
  codings.collapsed <- gsub("_", "", codings.collapsed)
  # split into individual signs
  signs <- unlist(strsplit(codings.collapsed, ""))
  # get data frame with frequency counts
  signs.df <- as.data.frame(table(signs))
  # order by frequency
  signs.df <- signs.df[order(-signs.df$Freq), ]
  # add column with site_name
  signs.df$material <- rep(material, nrow(signs.df))
  # append results to dataframe
  materialSigns.df <- rbind(materialSigns.df, signs.df)
}

```

Barplot for frequency counts per sign type and cave site.

```

materialSigns.barplot <- ggplot(materialSigns.df, aes(x = reorder(signs, Freq),
                                                         y = Freq, fill = signs)) +

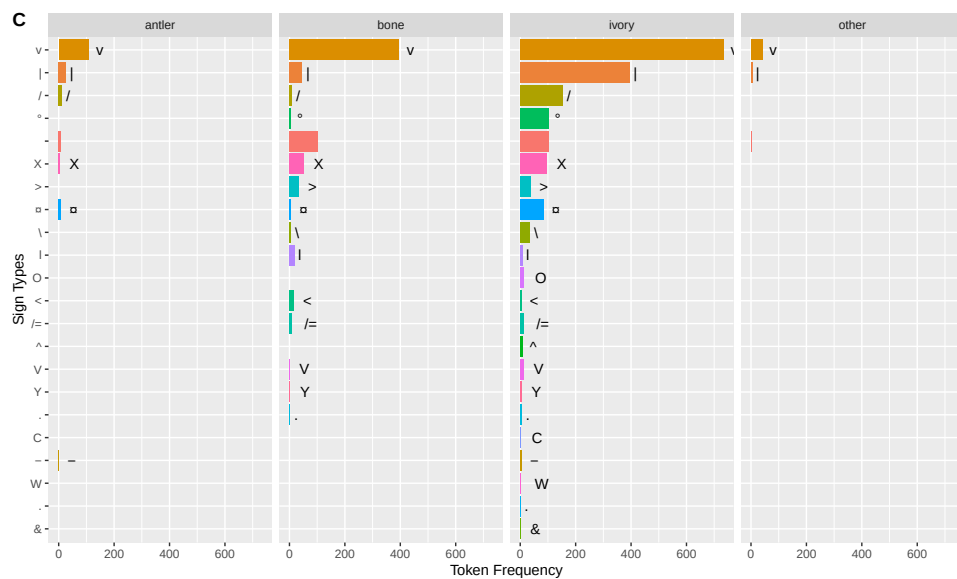
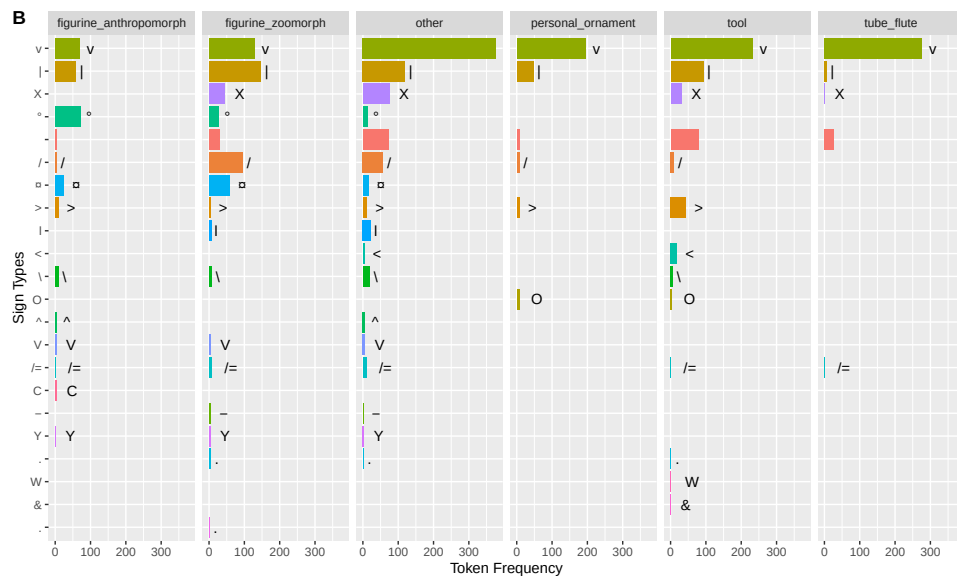
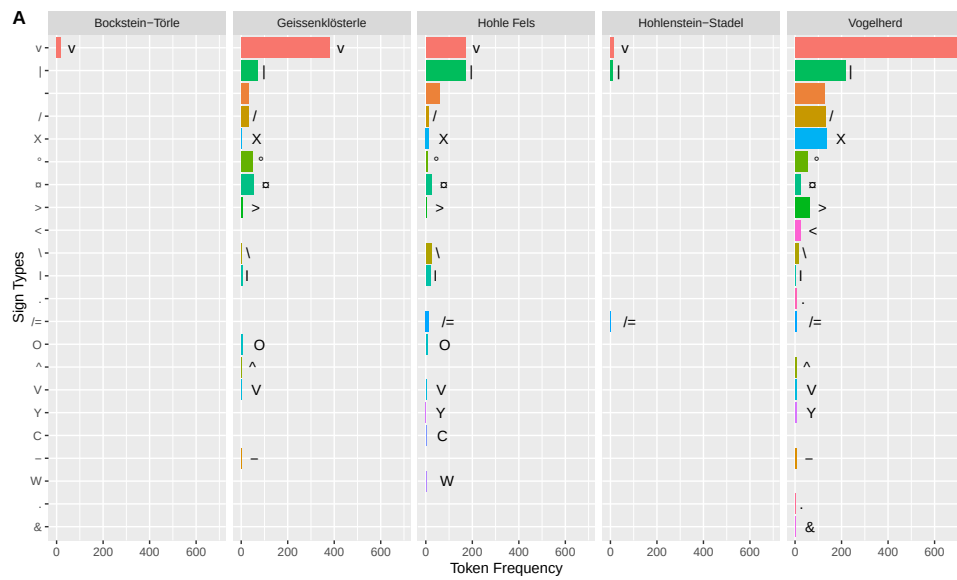
  #ylim(0, 700) +
  geom_text(aes(label = signs), hjust = -1) +
  geom_bar(stat = "identity") +
  facet_wrap(~ material, ncol = 4) +
  theme(legend.position = "none") +
  coord_flip() +
  xlab("Sign Types") +
  ylab("Token Frequency")
#materialSigns.barplot

```

Figures Combined

Combine figures.

```
signsCombined.barplot <- ggarrange(caveSigns.barplot, objectSigns.barplot, materialSigns.barplot,  
                                   nrow = 3,  
                                   labels = "AUTO")  
signsCombined.barplot
```



Safe complete figure to file.

```
ggsave("~/Github/PaleoSigns/figures/signsCombinedPlot_Aurignacian.pdf",
        signsCombined.barplot, dpi = 300, width = 10, height = 14, device = cairo_pdf)
```

Properties of Objects

Select Statistical Feature

```
# select the statistical feature for further analyses
feature.name <- "hrate.chars"
feature.values <- get(feature.name, sign.data)
```

Object Type (Carrier)

```
# create plot
objectType.plot <- ggplot(sign.data, aes(x = feature.values,
                                          y = fct_reorder(object_type, feature.values),
                                          fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                              quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = feature.name, option = "D") +
  geom_point(shape = 124, alpha = 0.7, size = 3) +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  ylab("Object-Type (Carrier)") +
  theme(legend.position = "none")
#objectType.plot
```

Material

```
# create plot
material.plot <- ggplot(sign.data, aes(x = feature.values,
                                         y = fct_reorder(material, feature.values),
                                         fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                              quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = feature.name, option = "D") +
  geom_point(shape = 124, alpha = 0.7, size = 3) +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  ylab("Material") +
  theme(legend.position = "none")
#material.plot
```

Cave Site

```
# create plot
site.plot <- ggplot(sign.data, aes(x = feature.values,
                                    y = fct_reorder(site_name, feature.values),
```

```

                                fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                                quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = feature.name, option = "D") +
  geom_point(shape = 124, alpha = 0.7, size = 3) +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  ylab("Cave Site") +
  theme(legend.position = "none")
#site.plot

```

Preservation

```

# create plot
preservation.plot <- ggplot(sign.data, aes(x = feature.values,
                                             y = fct_reorder(preservation, feature.values),
                                             fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                                quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = feature.name, option = "D") +
  geom_point(shape = 124, alpha = 0.7, size = 3) +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  ylab("Preservation") +
  theme(legend.position = "none")
#preservation.plot

```

Volume of Object

```

# create plot
volume.plot <- ggplot(sign.data, aes(x = volume, y = feature.values,
                                       size = volume, color = volume)) +
  geom_point(alpha = 0.5) +
  #geom_text_repel(aes(label = object_type), nudge_x = 0.2) +
  geom_smooth(method = "lm") +
  geom_smooth(method = "loess", linetype = "dashed") +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  ylab("Volume (cm^3)") +
  theme(legend.position = "none")
#volume.plot

```

Date

```

# create plot
date.plot <- ggplot(sign.data, aes(x = date, y = feature.values,
                                     size = volume, color = volume)) +
  geom_point(alpha = 0.5) +
  scale_x_continuous(trans = "reverse") +
  #geom_label_repel(aes(label = object_type)) +
  geom_smooth(method = "lm") +
  #geom_smooth(method = "loess", linetype = "dashed") +

```

```

ylab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
xlab("Date (BP, max)") +
theme(legend.position = "none")
#date.plot

```

Number of Sign Tokens

```

# create plot
token.plot <- ggplot(sign.data, aes(x = num.tokens, y = feature.values,
                                     size = volume, color = volume)) +

  geom_point(alpha = 0.5) +
  #geom_label_repel(aes(label = object_type)) +
  geom_smooth(method = "lm") +
  geom_smooth(method = "loess", linetype = "dashed") +
  ylab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  xlab("Number of Sign Tokens") +
  theme(legend.position = "none")
#token.plot

```

Number of Sign Types

```

# create plot
type.plot <- ggplot(sign.data, aes(x = num.types, y = feature.values,
                                    size = volume, color = volume)) +

  geom_point(alpha = 0.5) +
  #geom_label_repel(aes(label = object_type)) +
  geom_smooth(method = "lm") +
  geom_smooth(method = "loess", linetype = "dashed") +
  ylab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  xlab("Number of Sign Types") +
  theme(legend.position = "none")
#type.plot

```

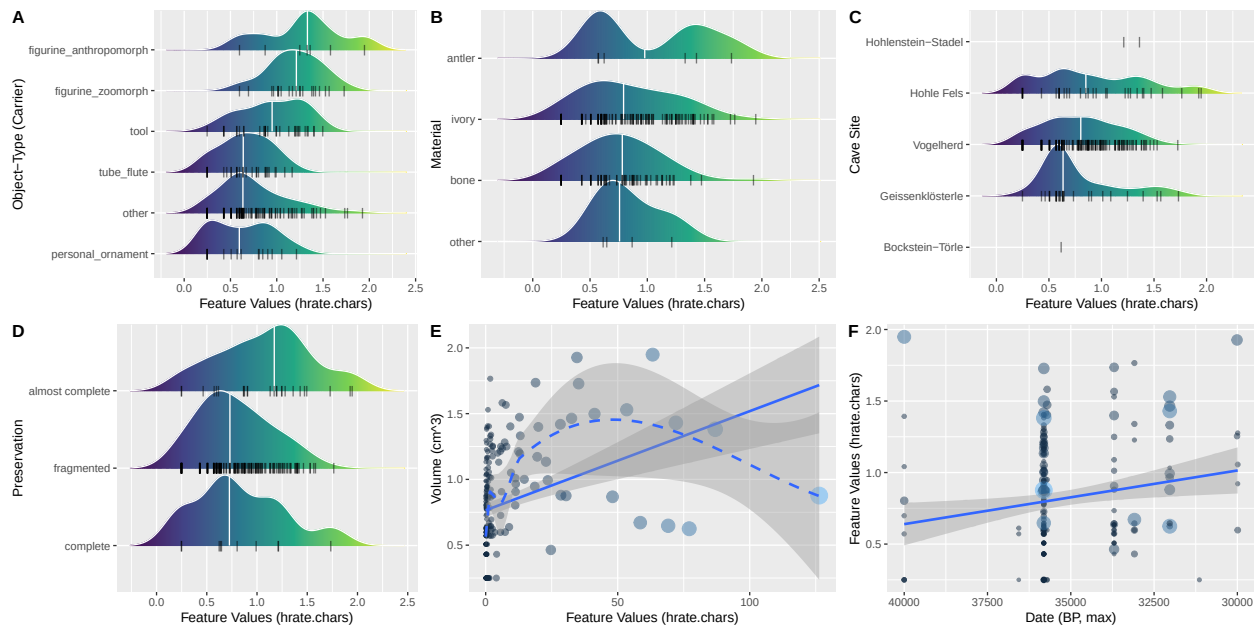
Figures Combined

Combine figures.

```

predictor.plot <- ggarrange(objectType.plot, material.plot, site.plot,
                             preservation.plot, volume.plot, date.plot,
                             ncol = 3, nrow = 2,
                             labels = "AUTO")
predictor.plot

```



Safe figure to file.

```
ggsave("~/Github/PaleoSigns/figures/predictorPlot_Aurignacian.pdf",
        predictor.plot, dpi = 300, width = 14, height = 7, device = cairo_pdf)
```

Statistical Model

Linear Models

We here fit linear models with feature values (e.g. entropy rates) as response variable, and meta information (e.g. volume, type of object, material etc.) as predictor variables. Alternatively, especially the number of different sign types could also be used as a response, as this is highly correlated with the entropy rate. However, model checking reveals that some of the assumptions of linear regression (e.g. normality of residuals) are not met in this case. Some of the predictors are numerical (e.g. volume) while others are categorical (e.g. material, object_type). Arguably, a linear multiple regression model is appropriate since each object and its respective sequence of signs can be seen as an independent data point. Of course, it is theoretically possible that the same paleolithic person has produced a subset of the objects. However, there is no way of knowing this with certainty. Hence, there is no clear evidence for “repeated measures” as in controlled experiments with humans (with subjects and items, cf. Baayen, 2013) – which would require a mixed-effects structure.

We here fit models stepwise by adding predictors one at a time on top of a baseline model (intercept only). We follow the advice by Baayen (2013) to use AIC as a criterion for model selection. If AIC decreases when a predictor is added, we leave the predictor in the model. We also consider interactions between object type and date as well as volume. It is theoretically conceivable that for cultures of the early Aurignacien (c. 40 000 years ago) object types might have been associated with information density (entropy rates) in a different way than for later Aurignacien cultures (c. 30 000 years ago). Similarly, volumes of objects might have played different roles in earlier compared to later periods.

Baseline Model

```
# model code
baseline.model <- lm(feature.values ~ 1, sign.data)
# model specification
formula.baseline <- "intercept"
# model summary
baseline.summary <- summary(baseline.model)
# get model information from summary
df.baseline <- baseline.summary$df[1]
res.df.baseline <- baseline.summary$df[2]
f.statistic.baseline <- NA
p.value.baseline <- NA
aic.baseline <- AIC(baseline.model)
```

Models with predictors added sequentially

Volume

```
# model code
model1 <- lm(feature.values ~ volume, sign.data)
model1.summary <- summary(model1)
formula.model1 <- "intercept + volume"
# get model information
df.model1 <- model1.summary$df[1]
res.df.model1 <- model1.summary$df[2]
f.statistic.model1 <- model1.summary$fstatistic[[1]]
p.value.model1 <- unname(pf(model1.summary$fstatistic[1L],
                           model1.summary$fstatistic[2L],
                           model1.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model1 <- AIC(model1)
```

Volume + Date (age)

```
# model code
model2 <- lm(feature.values ~ volume + date, sign.data)
model2.summary <- summary(model2)
formula.model2 <- "intercept + volume + date"
# get model information
df.model2 <- model2.summary$df[1]
res.df.model2 <- model2.summary$df[2]
f.statistic.model2 <- model2.summary$fstatistic[[1]]
p.value.model2 <- unname(pf(model2.summary$fstatistic[1L],
                           model2.summary$fstatistic[2L],
                           model2.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model2 <- AIC(model2)
```

Volume + Date (age) + Object type

```
model3 <- lm(feature.values ~ volume + date + object_type, sign.data)
model3.summary <- summary(model3)
formula.model3 <- "intercept + volume + date + object_type"
# get model information
df.model3 <- model3.summary$df[1]
res.df.model3 <- model3.summary$df[2]
f.statistic.model3 <- model3.summary$fstatistic[[1]]
p.value.model3 <- unname(pf(model3.summary$fstatistic[1L],
                           model3.summary$fstatistic[2L],
                           model3.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model3 <- AIC(model3)
```

Volume + Date (age) + Object type + Cave Site

```
model4 <- lm(feature.values ~ volume + date + object_type + site_name, sign.data)
formula.model4 <- "intercept + volume + date + object_type + site_name"
model4.summary <- summary(model4)
# get model information
df.model4 <- model4.summary$df[1]
res.df.model4 <- model4.summary$df[2]
f.statistic.model4 <- model4.summary$fstatistic[[1]]
p.value.model4 <- unname(pf(model4.summary$fstatistic[1L],
                           model4.summary$fstatistic[2L],
                           model4.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model4 <- AIC(model4)
```

Volume + Date (age) + Object type + Preservation

```
model5 <- lm(feature.values ~ volume + date + object_type + preservation, sign.data)
formula.model5 <- "intercept + volume + date + object_type + preservation"
model5.summary <- summary(model5)
# get model information
df.model5 <- model5.summary$df[1]
res.df.model5 <- model5.summary$df[2]
f.statistic.model5 <- model5.summary$fstatistic[[1]]
p.value.model5 <- unname(pf(model5.summary$fstatistic[1L],
                           model5.summary$fstatistic[2L],
                           model5.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model5 <- AIC(model5)
```

Volume + Date (age) + Object type + Material

```
model6 <- lm(feature.values ~ volume + date + object_type + material, sign.data)
formula.model6 <- "intercept + volume + date + object_type + material"
```

```

model6.summary <- summary(model6)
# get model information
df.model6 <- model6.summary$df[1]
res.df.model6 <- model6.summary$df[2]
f.statistic.model6 <- model6.summary$fstatistic[[1]]
p.value.model6 <- unname(pf(model6.summary$fstatistic[1L],
                           model6.summary$fstatistic[2L],
                           model6.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model6 <- AIC(model6)

```

Volume*Preservation + Date (age) + Object type

```

model7 <- lm(feature.values ~ volume*preservation + date + object_type, sign.data)
formula.model7 <- "intercept + volume*preservation + date + object_type"
model7.summary <- summary(model7)
# get model information
df.model7 <- model7.summary$df[1]
res.df.model7 <- model7.summary$df[2]
f.statistic.model7 <- model7.summary$fstatistic[[1]]
p.value.model7 <- unname(pf(model7.summary$fstatistic[1L],
                           model7.summary$fstatistic[2L],
                           model7.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model7 <- AIC(model7)

```

Volume*Object type + Date (age)

```

model8 <- lm(feature.values ~ volume*object_type + date, sign.data)
formula.model8 <- "intercept + volume*object_type + date"
model8.summary <- summary(model8)
# get model information
df.model8 <- model8.summary$df[1]
res.df.model8 <- model8.summary$df[2]
f.statistic.model8 <- model8.summary$fstatistic[[1]]
p.value.model8 <- unname(pf(model8.summary$fstatistic[1L],
                           model8.summary$fstatistic[2L],
                           model8.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model8 <- AIC(model8)

```

Volume + Date (age)*Object type

```

model9 <- lm(feature.values ~ volume + date*object_type, sign.data)
formula.model9 <- "intercept + volume + date*object_type"
model9.summary <- summary(model9)
# get model information
df.model9 <- model9.summary$df[1]
res.df.model9 <- model9.summary$df[2]

```

```
f.statistic.model9 <- model9.summary$fstatistic[[1]]
p.value.model9 <- unname(pf(model9.summary$fstatistic[1L],
                           model9.summary$fstatistic[2L],
                           model9.summary$fstatistic[3L],
                           lower.tail = FALSE))
aic.model9 <- AIC(model9)
```

Combine results in a table

```
# concatenate results per model
model.formula <- c(formula.baseline, formula.model1, formula.model2,
                  formula.model3, formula.model4, formula.model5,
                  formula.model6, formula.model7, formula.model8,
                  formula.model9)
df <- c(df.baseline, df.model1, df.model2, df.model3, df.model4, df.model5,
        df.model6, df.model7, df.model8, df.model9)
res.df <- c(res.df.baseline, res.df.model1, res.df.model2, res.df.model3,
            res.df.model4, res.df.model5, res.df.model6, res.df.model7,
            res.df.model8, res.df.model9)
f.statistic <- c(f.statistic.baseline, f.statistic.model1, f.statistic.model2,
                f.statistic.model3, f.statistic.model4, f.statistic.model5,
                f.statistic.model6, f.statistic.model7, f.statistic.model8,
                f.statistic.model9)
p.value <- c(p.value.baseline, p.value.model1, p.value.model2, p.value.model3,
             p.value.model4, p.value.model5, p.value.model6, p.value.model7,
             p.value.model8, p.value.model9)
aic <- c(aic.baseline, aic.model1, aic.model2, aic.model3, aic.model4,
         aic.model5, aic.model6, aic.model7, aic.model8, aic.model9)

# create data frame
model.results <- data.frame(model.formula, df, res.df, f.statistic, p.value, aic)
print(model.results)
```

```
##              model.formula df res.df f.statistic
## 1                intercept  1    210          NA
## 2            intercept + volume  2    205    23.952030
## 3    intercept + volume + date  3    201    14.127252
## 4    intercept + volume + date + object_type  8    196    10.117546
## 5    intercept + volume + date + object_type + site_name 10    194     8.140555
## 6    intercept + volume + date + object_type + preservation 10    194     8.601352
## 7    intercept + volume + date + object_type + material 11    193     7.385119
## 8    intercept + volume*preservation + date + object_type 12    192     7.229267
## 9                intercept + volume*object_type + date 13    191     6.584047
## 10    intercept + volume + date*object_type 13    191     6.462712
##      p.value      aic
## 1          NA 198.0151
## 2 1.996567e-06 174.2096
## 3 1.816592e-06 170.1340
## 4 8.656271e-11 144.0377
## 5 3.168924e-10 145.6076
## 6 8.133163e-11 142.4664
## 7 6.912348e-10 146.8694
## 8 2.872204e-10 144.2699
## 9 8.275178e-10 146.3449
## 10 1.301618e-09 147.4479
```

Export data frame with results of linear models.

```
write.csv(model.results, file = "~/Github/PaleoSigns/tables/lmResults.csv", row.names = F)
```

Manually select best model for further usage. Note that a decrease in AIC (compared to previous best model) is here considered as criterion for leaving the respective predictor variable in the model.

```
best.model <- model5
```

Model Summary

```
summary(best.model)
```

```
##
## Call:
## lm(formula = feature.values ~ volume + date + object_type + preservation,
##     data = sign.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.73937 -0.22832 -0.05451  0.22771  0.99876
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                1.701e+00  4.838e-01   3.516 0.000546 ***
## volume                     3.129e-03  1.681e-03   1.861 0.064245 .
## date                      -1.808e-05  1.348e-05  -1.341 0.181609
## object_typefigurine_anthropomorph  2.784e-01  1.489e-01   1.871 0.062914 .
## object_typefigurine_zoomorph      2.163e-01  9.443e-02   2.291 0.023036 *
## object_typetube_flute            -2.380e-01  8.779e-02  -2.711 0.007308 **
## object_typepersonal_ornament     -3.282e-01  1.173e-01  -2.799 0.005645 **
## object_typeother                -1.489e-01  6.894e-02  -2.159 0.032041 *
## preservationcomplete            -1.732e-01  1.333e-01  -1.300 0.195277
## preservationfragmented          -1.928e-01  8.453e-02  -2.281 0.023648 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3334 on 194 degrees of freedom
## (7 observations deleted due to missingness)
## Multiple R-squared:  0.2852, Adjusted R-squared:  0.2521
## F-statistic: 8.601 on 9 and 194 DF, p-value: 8.133e-11
```

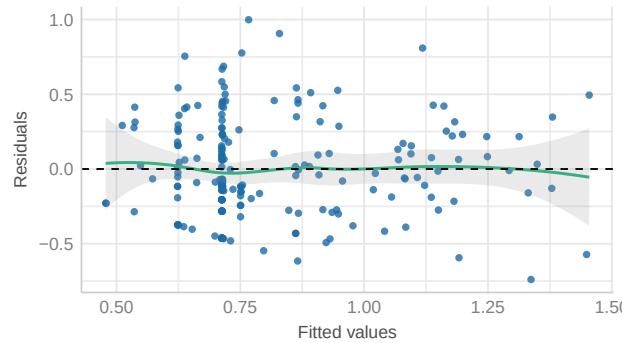
Interpretation: In the model summary above, the reference level for object_type is “tools”, i.e. the residue category. Hence, the entropy rates of sign sequences found on anthropomorph figurines are on average *higher* than the ones found on objects classified as tools. A similar effect occurs for zoomorph figurines which also carry more information than tools on average. Inversely, entropy rates of sequences on tubes and flutes as well as personal ornaments are somewhat *lower* than on tools. There is no significant effect of the volume of an object on the entropy rate (though the coefficient is positive, as one would expect). There is no significant effect of the age of an object (c. 30 000 to 40 000 years old) either. Lastly, fragmented objects tend to have lower entropy rate than “almost complete” ones. The details of the results (e.g. p-values) change for different versions of preprocessing (i.e. including and excluding certain data points), but the overall trends are similar. In particular, the entropy rate of sign sequences on zoomorph figurines is robustly higher than for the reference level of tools.

Checking of Model Assumptions

```
check_model(best.model)
```

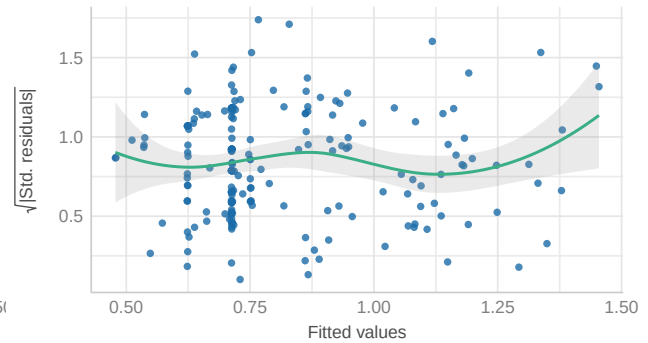
Linearity

Reference line should be flat and horizontal



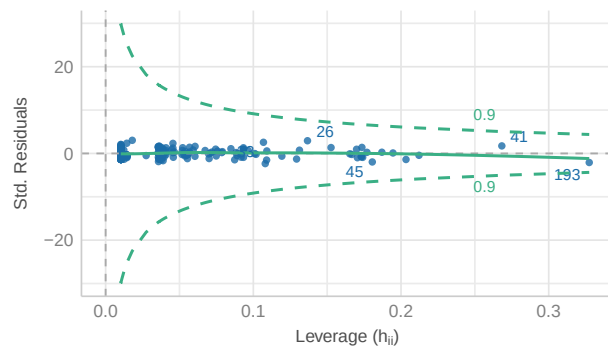
Homogeneity of Variance

Reference line should be flat and horizontal



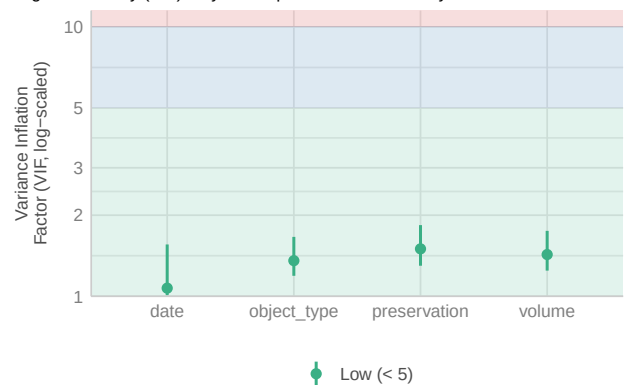
Influential Observations

Points should be inside the contour lines



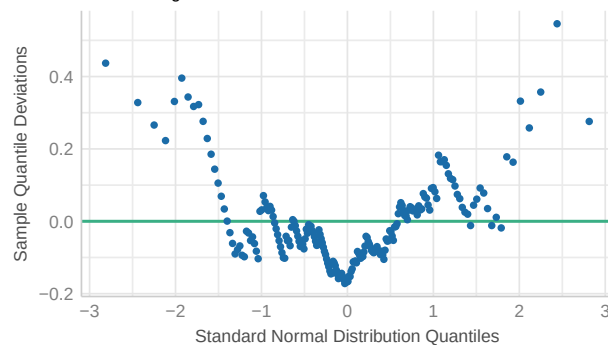
Collinearity

High collinearity (VIF) may inflate parameter uncertainty



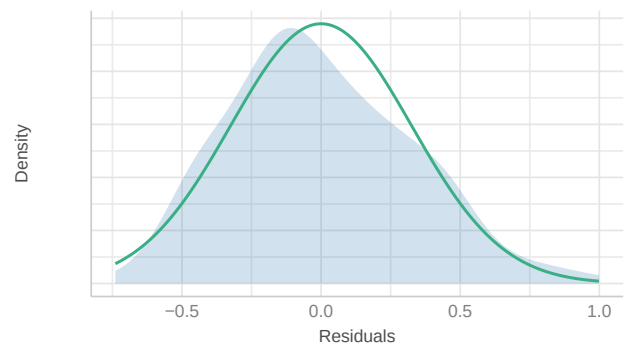
Normality of Residuals

Dots should fall along the line



Normality of Residuals

Distribution should be close to the normal curve



Write to file.

```
png(filename = file.path("~/Github/PaleoSigns/figures/", "modelCheckingPlot.png"),
     width = 10, height = 10, units = "in", res = 300)
check_model(best.model)
dev.off()
```

```
## pdf
## 2
```

Shapiro-Wilk Test for Normality of Residuals

```
shapiro.test(rstandard(best.model))
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  rstandard(best.model)  
## W = 0.9844, p-value = 0.02354
```

Interpretation: There is a slight deviation from homogeneity of variance for the residuals (upper right panel), though this is due to just one or two data points deviating for the highest fitted values. There are no overly influential observations, and no collinearity between predictors (middle panels). There is a slight skew visible in the distribution of the residuals (lower panels). The Shapiro-Wilk test for normality gives a p-value around 0.05, meaning that the distribution does slightly deviate from the expectation of a normal distribution (null hypothesis).

Plot Linear Model Coefficients

```
coefficient.plot <- plot_coefs(best.model,  
                              plot.distributions = F,  
                              rescale.distributions = F,  
  coefs = c("Object type: figurine anthropomorph" = "object_typefigurine_anthropomorph",  
            "Object type: figurine zoomorph" = "object_typefigurine_zoomorph",  
            "Object type: tube or flute" = "object_tubetube_flute",  
            "Object type: tool" = "object_typetool",  
            "Object type: personal ornament" = "object_typepersonal_ornament",  
            "Object type: other" = "object_typeother",  
            "Material: antler" = "materialantler",  
            "Material: bone" = "materialbone",  
            "Material: ivory" = "materialivory",  
            "Material: other" = "materialother",  
            "Volume" = "volume",  
            "Date (age of object)" = "date",  
            "Preservation: complete" = "preservationcomplete",  
            "Preservation: fragmented" = "preservationfragmented",  
            "Site name: Hohle Fels" = "site_nameHohle Fels",  
            "Site name: Vogelherd" = "site_nameVogelherd"))
```

```
## Loading required namespace: broom.mixed
```

```
#coefficient.plot
```

Sequence plot

```
# exclude "other" category (otherwise the sequence plot is overcrowded)  
sign.data.modified <- sign.data[sign.data$object_type != "other", ]  
# take random subsample as otherwise the sequence plot is overcrowded  
set.seed(09012020)  
sign.data.subsample <- sample_n(sign.data.modified, 50)  
# get feature values for subsample
```

```

feature.values.modified <- get(feature.name, sign.data.modified)
feature.values.subsample <- get(feature.name, sign.data.subsample)

# create plot
sequence.plot <- ggplot(sign.data.subsample, aes(x = feature.values.subsample,
                                                  y = reorder(paste(coding_clean, object_id, sep = " "),
                                                                feature.values.subsample, mean),
                                                  size = volume, color = object_type)) +

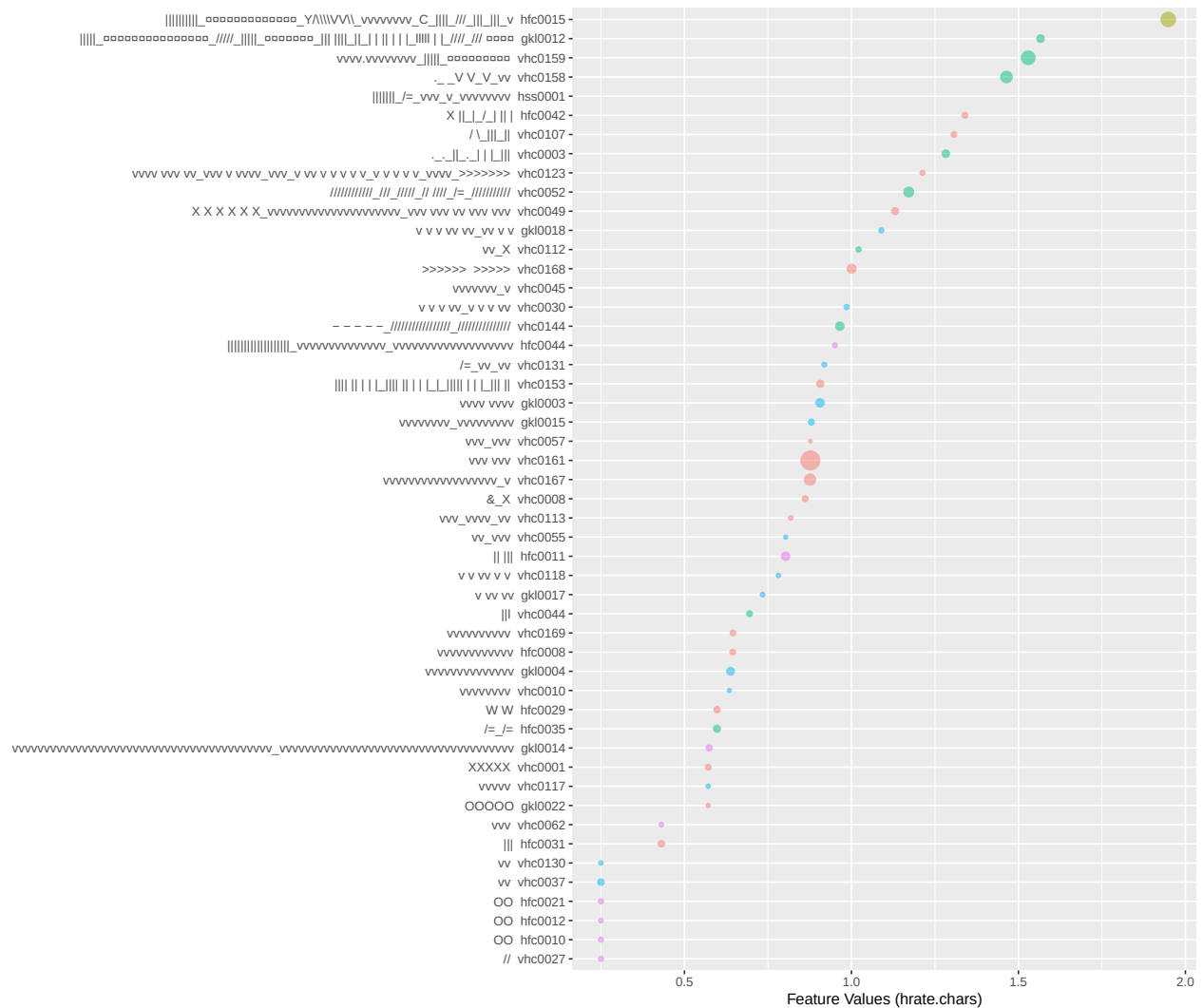
  geom_point(alpha = 0.5) +
  ylab("") +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  theme(legend.position = "none")
sequence.plot

```

```

## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).

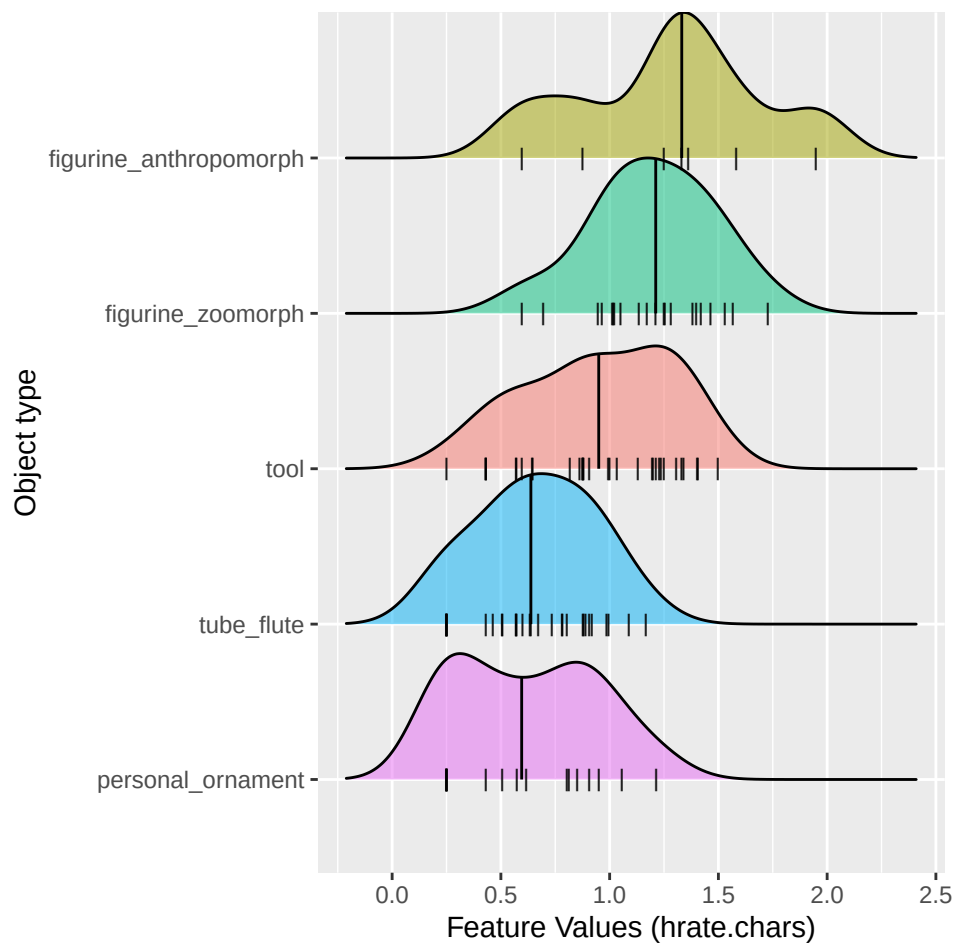
```



Density plot

```
density.plot <- ggplot(sign.data.modified, aes(x = feature.values.modified,
                                              y = fct_reorder(object_type, feature.values.modified),
                                              fill = object_type)) +
  geom_density_ridges(alpha = 0.5, scale = 1, quantile_lines = T,
                    quantiles = 2) +
  geom_point(shape = 124, alpha = 0.9, size = 3) +
  ylab("Object type") +
  xlab(paste("Feature Values ", "(", feature.name, ")", sep = "")) +
  theme(legend.position = "none")
density.plot
```

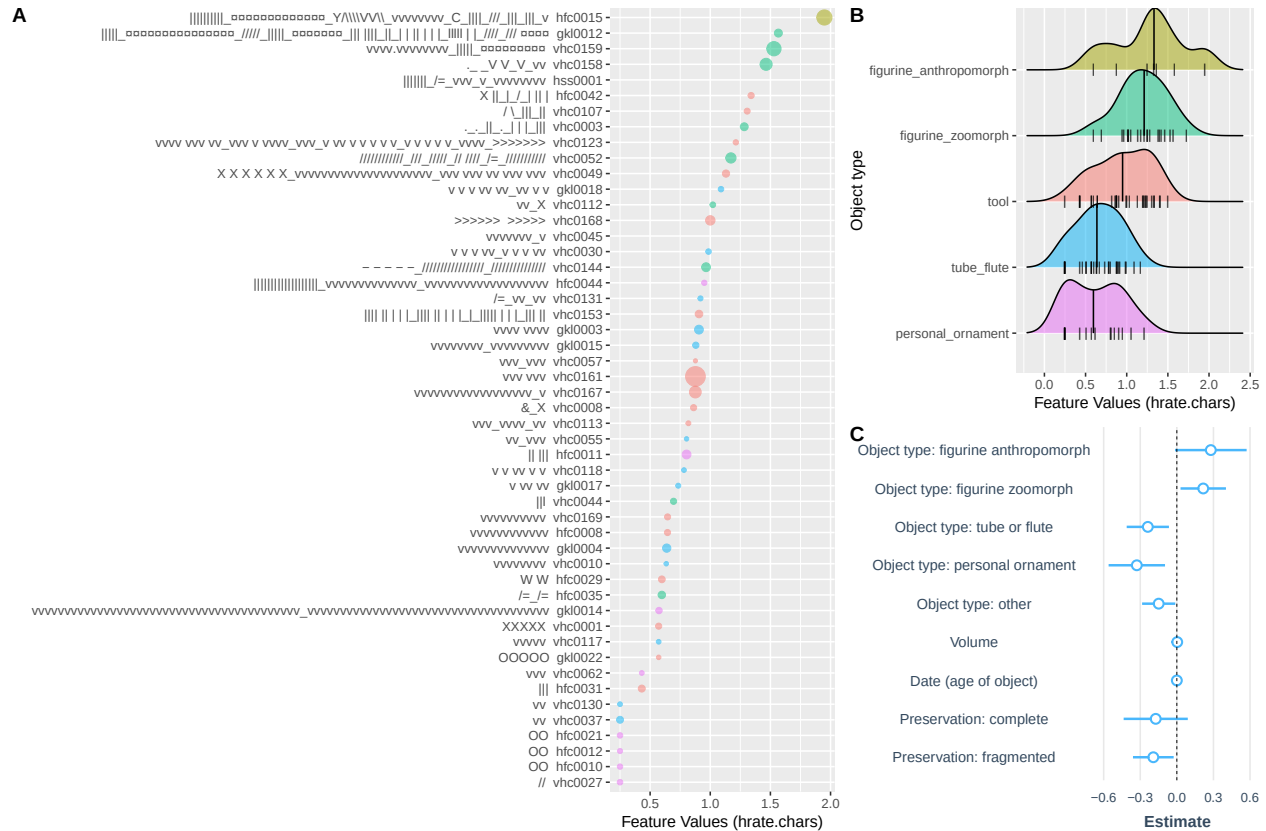
Picking joint bandwidth of 0.154



Combine figures.

```
model.plot <- ggarrange(sequence.plot,                                # First column with sequence plot
                        ggarrange(density.plot, coefficient.plot,
                                ncol = 1, labels = c("B", "C")), # Second column with density and coefficient plot
                        ncol = 2,
                        labels = "A",
                        widths = c(2,1)                             # Relative widths of panels
                        )
```

model.plot



Safe figure to file.

```
ggsave("~/Github/PaleoSigns/figures/modelPlot_Aurignacian.pdf",
        model.plot, dpi = 300, width = 12, height = 8, device = cairo_pdf)
```

Package References

```
## Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_.
## Springer-Verlag New York. ISBN 978-3-319-24277-4,
## <https://ggplot2.tidyverse.org>.

## Slowikowski K (2024). _ggrepel: Automatically Position Non-Overlapping
## Text Labels with 'ggplot2'_. R package version 0.9.5,
## <https://CRAN.R-project.org/package=ggrepel>.

## Attali D, Baker C (2023). _ggExtra: Add Marginal Histograms to
## 'ggplot2', and More 'ggplot2' Enhancements_. R package version 0.10.1,
## <https://CRAN.R-project.org/package=ggExtra>.

## Auguie B (2017). _gridExtra: Miscellaneous Functions for "Grid"
## Graphics_. R package version 2.3,
## <https://CRAN.R-project.org/package=gridExtra>.

## Kassambara A (2023). _ggpubr: 'ggplot2' Based Publication Ready Plots_.
## R package version 0.6.0, <https://CRAN.R-project.org/package=ggpubr>.
```

```
## Wilke C (2024). _ggridges: Ridgeline Plots in 'ggplot2'_. R package
## version 0.5.6, <https://CRAN.R-project.org/package=gggridges>.

## Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R,
## Golemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E,
## Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi
## K, Vaughan D, Wilke C, Woo K, Yutani H (2019). "Welcome to the
## tidyverse." _Journal of Open Source Software_, *4*(43), 1686.
## doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.

## Lüdecke D, Ben-Shachar M, Patil I, Waggoner P, Makowski D (2021).
## "performance: An R Package for Assessment, Comparison and Testing of
## Statistical Models." _Journal of Open Source Software_, *6*(60), 3139.
## doi:10.21105/joss.03139 <https://doi.org/10.21105/joss.03139>.

## Lüdecke D, Patil I, Ben-Shachar M, Wiernik B, Waggoner P, Makowski D
## (2021). "see: An R Package for Visualizing Statistical Models."
## _Journal of Open Source Software_, *6*(64), 3393.
## doi:10.21105/joss.03393 <https://doi.org/10.21105/joss.03393>.

## Venables WN, Ripley BD (2002). _Modern Applied Statistics with S_,
## Fourth edition. Springer, New York. ISBN 0-387-95457-0,
## <https://www.stats.ox.ac.uk/pub/MASS4/>.

## Long JA (2022). _jtools: Analysis and Presentation of Social Scientific
## Data_. R package version 2.2.0,
## <https://cran.r-project.org/package=jtools>.
```

Literature References

- Baayen, R. H. (2008). *Analyzing Linguistic Data. A Practical Introduction to Statistics Using R*. Cambridge: Cambridge University Press.
- Baayen, H. (2013). Multivariate Statistics. In: Podesva, R. J. & Sharma, D. (eds.), *Research methods in linguistics*. Cambridge: Cambridge University Press, p. 337-372.